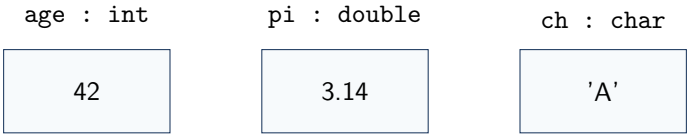
# Типы, переменные, I/O и <cmath>

Введение в ПИ (C++)

Types, Variables, I/O, Math

# Переменная = имя + тип + значение




Тип задаёт **множество значений** и **допустимые операции**.

# Частые типы — примеры

```cpp
#include <iostream>
using namespace std;

int main() {
    int credits = 30;           // whole number
    double gpa = 3.75;          // floating point
    bool enrolled = true;       // true / false
    char letter = 'B';          // one character
    long long id = 2026001234LL;

    cout << credits << " cr, GPA " << gpa << "\n";
    cout << "letter=" << letter
         << " enrolled=" << enrolled << "\n";
}
```

Размер `int` зависит от платформы; для больших целых — `long long`.

# Ещё примеры объявлений

```
int n = 0;
int sum{};                  // zero-initialized
double price = 49.90;
double tax = price * 0.12;
bool ok = (tax > 0);
char mark = 'A';
```

Имена: понятные (`price`, `tax`), не `x1`/`tmp2` без причины.

# Инициализация и const

```cpp
int x = 5;                  // copy init
int y{5};                   // brace init (preferred)
const int MAX_SCORE = 100;
const double PI = 3.141592653589793;
// MAX_SCORE = 200;         // compile error
```

**Запомните**

`const` — обещание не менять; компилятор помогает держать слово.

# Арифметика — живой пример

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2;
    cout << a + b << " " << a - b << "\n";   // 9 5
    cout << a * b << " " << a / b << "\n";   // 14 3   (int!)
    cout << a % b << "\n";                   // 1
    cout << 7.0 / 2 << "\n";                 // 3.5
}
```

# Преобразования типов

```cpp
int whole = 5;
double x = whole;            // int -> double (widening)
double y = 3.9;
int truncated = y;           // 3  (fraction dropped)
int z = static_cast<int>(3.9);  // explicit: 3
```

**Частая ошибка**

Сравнивать `double` через `==` после арифметики — осторожно с погрешностью.

# Переполнение *(overflow)* — идея

У фиксированной ширины целого есть максимум.

Пример порядка: у типичного 32-bit `int` максимум $\approx 2 \cdot 10^9$.

При переполнении signed-поведение в стандартном C++ — **undefined behavior** (не полагайтесь на «обернётся»).

# Заголовок <cmath>

```cpp
#include <cmath>
using namespace std;    // then: sqrt, pow, fabs, ...
```

Математические функции стандартной библиотеки *(standard library math functions)*.
В примерах курса после `using namespace std;` вызываем `sqrt`, `pow`, ... без префикса.

# Частые функции

| Функция | Смысл | Пример |
|---|---|---|
| `sqrt(x)` | $\sqrt{x}$ | `sqrt(9.0)` $\to$ `3.0` |
| `pow(a,b)` | $a^b$ | `pow(2.0, 3.0)` $\to$ `8.0` |
| `fabs(x)` | $\lvert x \rvert$ для `double` | `fabs(-2.5)` $\to$ `2.5` |
| `abs(x)` | модуль (`int`/...) | `abs(-7)` $\to$ `7` |
| `ceil(x)` | вверх до целого | `ceil(2.1)` $\to$ `3.0` |
| `floor(x)` | вниз до целого | `floor(2.9)` $\to$ `2.0` |

Тригонометрия (легко): `sin`, `cos` — аргумент в **радианах**.

# Пример: гипотенуза и округление

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double a = 3.0, b = 4.0;
    double c = sqrt(a * a + b * b);   // or pow(a, 2)
    cout << c << "\n";                // 5
    cout << ceil(2.1) << "\n";        // 3
    cout << floor(2.9) << "\n";       // 2
}
```

# Пример: скидка и модуль

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double price = 120.0;
    double discount = 15.5;
    double pay = price - discount;
    cout << fabs(pay - 100.0) << "\n";  // how far from 100
    cout << ceil(pay) << " " << floor(pay) << "\n";
}
```

# abs vs fabs

```cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n = -3;
    double x = -2.5;
    cout << abs(n) << "\n";     // 3   (int)
    cout << fabs(x) << "\n";    // 2.5
}
```

**Частая ошибка**

Для `double` надёжнее `fabs` (или перегрузки `abs` из `<cmath>`).

# Замечания

- `sqrt` от отрицательного — не число *(NaN)* / домен-ошибка; проверяйте вход
- `pow(a, 2)` vs `a * a`: для квадрата часто проще умножение
- `sin`/`cos`: градусы $\rightarrow$ радианы: $\alpha \cdot \pi / 180$

# cout и cin

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name;
    int age{};
    cout << "Name and age: ";
    cin >> name >> age;
    cout << name << " is " << age << "\n";
}
```

# Ещё I/O: чек за обед

```cpp
#include <iostream>
using namespace std;

int main() {
    string item;
    double price;
    int qty;
    cin >> item >> price >> qty;
    cout << item << " total=" << price * qty << "\n";
}
```

# Потоки

- `«` — вставить в поток вывода
- `»` — извлечь из потока ввода (по пробелам/переводам строк)
- Для строки с пробелами: `getline(cin, line)` (после `using namespace std;`)

**Частая ошибка**

После `cin » x` перевод строки остаётся в буфере — мешает следующему `getline`.

# auto (осторожно на старте)

```cpp
auto n = 42;        // int
auto x = 3.14;      // double
auto ok = true;     // bool
```

Удобно, когда тип очевиден справа. Пока тип важен для обучения — пишите явно.

# Викторина 1 — тип значения

## Викторина

Какой тип у выражения `7 / 2` при целых операндах?

1. `double`, значение `3.5`
2. `int`, значение `3`
3. `float`, значение `3.5`
4. ошибка компиляции

*Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.*

# Ответ — Викторина 1

**Ответ**

**2.** Целочисленное деление: `7 / 2 == 3`.

Для `3.5`: `7.0 / 2` или `7 / 2.0`.

# Викторина 2 — const

## Викторина

Что произойдёт?

```
1 const int n = 10;
2 n = 11;
```

1. Напечатает 11
2. Ошибка компиляции
3. Ошибка только во время выполнения
4. Тихо проигнорирует присваивание

*Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.*

# Ответ — Викторина 2

**Ответ**

**2.** Присваивание `const`-переменной запрещено на этапе компиляции.

# Викторина 3 — cmath

## Викторина

Что нужно, чтобы вызвать `sqrt(9.0)` в таком стиле курса?

1. Ничего — функция всегда доступна
2. `#include <cmath>` и обычно `using namespace std;`
3. Только `#include <iostream>`
4. Только линкер без заголовка

*Пауза — ответьте (кликер / поднимите руку / сосед), затем следующий слайд.*

# Ответ — Викторина 3

**Ответ**

**2.** Объявления — в `<cmath>`; с `using namespace std;` можно писать `sqrt`.

(1)/(3) недостаточно; (4) без заголовка код не скомпилируется корректно.